IoT Weed Detection & Removal Robot

Phase 5 – Remaining Task Status by Team Member | August 13, 2026

Member 1 — Backend Developer

Done

- POST /api/detection/weed – validates payload, 4s cooldown, logs to logs.json (source: "ai-vision"), records into session tracking, triggers cutter via ESP32 service
- GET /api/detection/status – returns last detection info
- GET /api/detection/stream – proxies the live camera MJPEG feed to the frontend
- New sessionService.js – dedicated session read/write logic

Remaining / Open

- Confirm whether the cutter relay is meant to auto-turn-off via the ESP32 watchdog, or whether an explicit relay("off") call is still needed after a detection — code currently only turns it on
- Provide Postman collection or curl examples for the two new detection endpoints, promised to Member 4 for API documentation

Member 2 — Frontend Developer

Remaining (not yet seen in any shared upload)

- camera-feed.html – new page for the live stream + detection overlay
- camera-feed.js – poll GET /api/detection/status and render video/overlay
- camera-feed.css – styling consistent with existing dashboard pages
- Add a navigation link / card to the new Camera Feed page on index.html
- Add helper functions for the two new detection endpoints in api.js
- Confirm the exact JSON shape of GET /api/detection/status with Member 1 before wiring up rendering (per original coordination notes)

Member 3 — ESP32 / Vision Hardware Developer (You)

Done

- Dataset collected + labeled: 1,300 images (Kaggle crop-and-weed set), split 1040 / 130 / 130 train/val/test
- YOLOv8 model trained (yolov8s, 78 epochs): 82.7% precision / 73.5% recall – below the ≥85% target but usable for a full pipeline demo
- Model sanity-tested on held-out real images – detections visually correct
- sketch_phase5_vision.ino written (camera streaming firmware for the T-SIMCAM)
- detect_stream.py written (inference at ~1 FPS, confidence >50%, POSTs to backend)
- Firmware-level cutter safety interlock added (v7): rejects cutter-on requests with HTTP 409 while the robot is mid obstacle-avoidance
- Fixed a real endpoint contract mismatch: firmware updated from GET /cutter to POST /relay to match what the backend actually calls

Remaining

- T-SIMCAM hardware bring-up – blocked until the physical camera module is available
- Live-test detect_stream.py against the real camera stream once hardware arrives

- Confirm the two ESP32 boards (drive + camera) coexist cleanly on the same Wi-Fi network without IP/port conflicts
- Flash and verify the v7 safety interlock on real hardware (409 response not yet tested outside code review)
- Repo cleanup: delete duplicate dataset/labeled/agri_data/ folder, delete the 80MB software/ai-vision.zip archive, move/rename the firmware file to match the expected path (hardware/ESP-32/sketch_phase5_vision/sketch_phase5_vision.ino)

Member 4 — QA, Documentation & Integration

Remaining (not yet seen in any shared upload)

- docs/phase5-test-report.md – test log for detection accuracy, latency, and demo runs
- docs/dataset-collection-guide.md – documentation of how/where the dataset was sourced and labeled
- docs/Resources/Phase5_Plan.pdf – Phase 5 master plan document
- T-SIMCAM wiring/setup diagram alongside existing Phase 4 diagrams
- Updated component_tracker.xlsx with T-SIMCAM and other Phase 5 hardware line items
- Test plan: detection accuracy, end-to-end latency, false-positive rate, dual-ESP32 network stability
- End-to-end integration testing once Members 1/2/3's pieces are fully wired together

Note: this status reflects only what has been shared in project file uploads so far. Work sitting locally on a teammate's machine that hasn't been shared yet won't be reflected here — worth a quick check-in with Members 2 and 4 directly.